

AI Final Exam Revision Guide

1. Decision Trees

What It Is

A supervised learning algorithm that creates a tree-like model of decisions to predict target values by splitting data based on feature values.

How It Works

Structure:

- **Root Node:** Starting point with entire dataset
- **Internal Nodes:** Decision points (feature tests)
- **Branches:** Outcomes of decisions
- **Leaves:** Final predictions/classifications

Step-by-Step Process:

1. Start with entire dataset at root
2. Find best feature to split on (using metrics below)
3. Create branches for each possible value of that feature
4. Repeat recursively for each subset
5. Stop when pure nodes or stopping criteria met

Key Algorithms & Metrics

ID3 Algorithm:

- Uses Information Gain
- Only handles categorical features
- Prone to overfitting

C4.5 Algorithm:

- Improved version of ID3
- Handles continuous features
- Uses Gain Ratio to reduce bias

CART Algorithm:

- Uses Gini Impurity

- Creates binary splits only
- Handles both classification and regression

Essential Formulas

Entropy (E):

$$E(S) = -\sum(p_i \times \log_2(p_i))$$

- Where p_i = proportion of examples in class i
- Measures impurity/disorder in dataset

Information Gain:

$$IG(S,A) = E(S) - \sum((|S_v|/|S|) \times E(S_v))$$

- Where S_v = subset after splitting on attribute A

Gini Impurity:

$$Gini(S) = 1 - \sum(p_i^2)$$

- Alternative to entropy, computationally faster

Memory Aid: "SPLIT"

- **S**elect best feature (highest IG/lowest Gini)
- **P**artition data based on feature values
- **L**abel pure nodes as leaves
- **I**terate on remaining subsets
- **T**erminate when stopping criteria met

Pruning & Overfitting

- **Pre-pruning:** Stop early (min samples, max depth)
- **Post-pruning:** Build full tree, then remove branches
- Prevents overfitting to training data

Use Cases

- Medical diagnosis, credit approval, spam detection
- When interpretability is crucial
- Moderate-sized datasets with mixed feature types

Strengths vs Weaknesses

✅ **Strengths:** Interpretable, handles mixed data, no assumptions about data distribution ❌

Weaknesses: Prone to overfitting, unstable (small data changes = different tree), biased toward features with more levels

2. Naïve Bayes

What It Is

A supervised probabilistic classifier based on Bayes' theorem with the "naïve" assumption that features are conditionally independent.

Core Theory: Bayes' Theorem

$$P(A|B) = P(B|A) \times P(A) / P(B)$$

For Classification:

$$P(\text{class}|\text{features}) = P(\text{features}|\text{class}) \times P(\text{class}) / P(\text{features})$$

How It Works

1. Calculate prior probabilities $P(\text{class})$ for each class
2. Calculate likelihood $P(\text{feature}|\text{class})$ for each feature given each class
3. For new instance, multiply prior $\times$ all likelihoods
4. Choose class with highest probability

Naïve Independence Assumption

Assumes features are independent given the class:

$$P(x_1, x_2, \dots, x_n | \text{class}) = P(x_1 | \text{class}) \times P(x_2 | \text{class}) \times \dots \times P(x_n | \text{class})$$

Three Main Variants

1. Gaussian Naïve Bayes

- For continuous features
- Assumes features follow normal distribution
- Formula: $P(x|\text{class}) = \frac{1}{\sqrt{2\pi\sigma^2}} \times e^{-(x-\mu)^2/2\sigma^2}$

2. Multinomial Naïve Bayes

- For discrete count data (text classification)
- Features represent frequency of occurrence
- Good for document classification

3. Bernoulli Naïve Bayes

- For binary features (0/1, True/False)
- Each feature represents presence/absence
- Good for text with binary word occurrence

Memory Aid: "BIAS"

- **B**ayes theorem foundation
- **I**ndependence assumption (naïve)
- **A**ll features contribute multiplicatively
- **S**elect class with maximum posterior probability

Use Cases

- Spam filtering, sentiment analysis, medical diagnosis
- Text classification and document categorization
- Real-time predictions (fast computation)

Strengths vs Weaknesses

✅ **Strengths:** Fast, works well with small datasets, handles multi-class naturally, good baseline ❌

Weaknesses: Strong independence assumption (rarely true), sensitive to skewed data, zero probability problem

When It Works Best

- High-dimensional data (curse of dimensionality doesn't affect it much)
- Categorical inputs
- Need for fast, simple solution
- When independence assumption roughly holds

3. Support Vector Machines (SVM) ✂️

What It Is

A supervised learning algorithm that finds the optimal hyperplane to separate classes by maximizing the margin between them.

Core Concepts

Hyperplane: Decision boundary that separates classes

- In 2D: line
- In 3D: plane
- In n-D: (n-1) dimensional hyperplane

Margin: Distance between hyperplane and nearest data points **Support Vectors:** Data points closest to hyperplane (on the margin)

How It Works (Linear SVM)

1. Find hyperplane that separates classes
2. Among all possible hyperplanes, choose one with maximum margin
3. Only support vectors matter for decision boundary
4. New points classified based on which side of hyperplane they fall

Mathematical Foundation

Hyperplane equation: $w \cdot x + b = 0$ **Classification:** $\text{sign}(w \cdot x + b)$ **Margin:** $2 / ||w||$

Objective: Maximize margin = Minimize $||w||^2 / 2$

Linear vs Non-Linear Classification

Linear SVM: Data is linearly separable

- Direct hyperplane in original space
- Fast and simple

Non-Linear SVM: Data not linearly separable

- Use kernel trick to map to higher dimensions
- Find linear boundary in transformed space

Kernel Trick 🧙

Transform data to higher dimensional space where it becomes linearly separable without explicitly computing the transformation.

Common Kernels:

- **Linear:** $K(x, y) = x \cdot y$
- **Polynomial:** $K(x, y) = (x \cdot y + c)^d$
- **RBF (Radial Basis Function):** $K(x, y) = e^{-\gamma ||x - y||^2}$

Soft Margin Classification

Real data often not perfectly separable → allow some misclassification

- **C parameter:** Controls trade-off between margin size and misclassification
- High C: Smaller margin, fewer misclassifications
- Low C: Larger margin, more misclassifications allowed

Memory Aid: "MARGIN"

- **M**aximize the margin between classes
- **A**llow some errors with soft margin (C parameter)
- **R**ely only on support vectors
- **G**o to higher dimensions with kernels
- **I**deal for binary classification
- **N**on-linear boundaries possible

Use Cases

- Text classification, image classification, bioinformatics
- High-dimensional data
- When you need robust, accurate classifier

Strengths vs Weaknesses

✅ **Strengths:** Effective in high dimensions, memory efficient (uses only support vectors), versatile (different kernels) ❌ **Weaknesses:** Slow on large datasets, sensitive to feature scaling, no probabilistic output, choice of kernel/parameters crucial

When SVM Works Best

- High-dimensional data
- Clear margin of separation exists
- More dimensions than samples
- Need for robust, accurate classifier

4. Convolutional Neural Networks (CNN)

What It Is

A supervised deep learning architecture specifically designed for processing grid-like data (images) using convolutional layers that detect local features.

Core Architecture Components

1. Convolutional Layer

- Applies filters/kernels to detect features
- **Filter/Kernel:** Small matrix that slides over input
- **Feature Map:** Output after applying filter
- Detects edges, textures, patterns

2. Pooling Layer

- Reduces spatial dimensions
- **Max Pooling:** Takes maximum value in region
- **Average Pooling:** Takes average value
- Provides translation invariance

3. Fully Connected Layer

- Traditional neural network layer
- Usually at the end for final classification
- Flattens feature maps to 1D vector

Key Concepts

Stride: Step size when moving filter

- Stride = 1: Move one pixel at a time
- Stride = 2: Skip every other pixel

Padding: Adding zeros around input

- **Valid:** No padding
- **Same:** Pad to keep output size same as input

How CNNs Work

1. **Feature Detection:** Conv layers detect low-level features (edges, corners)
2. **Feature Reduction:** Pooling reduces dimensionality
3. **Feature Combination:** Deeper layers combine features into complex patterns
4. **Classification:** FC layers make final prediction

Mathematical Operations

Convolution Operation:

$$(f * g)(x,y) = \sum_i \sum_j f(i,j) \times g(x-i, y-j)$$

Output Size After Convolution:

$$\text{Output} = (\text{Input} - \text{Filter} + 2 \times \text{Padding}) / \text{Stride} + 1$$

Memory Aid: "FILTER"

- Feature detection with convolution
- Invariance through pooling
- Layers get deeper, features get complex
- Translation invariant
- Edges → textures → objects → scenes
- Reduce parameters through weight sharing

Hierarchical Feature Learning

- **Layer 1:** Edges, lines
- **Layer 2:** Corners, contours
- **Layer 3:** Shapes, textures
- **Layer 4:** Object parts
- **Layer 5:** Full objects

Use Cases

- Image classification, object detection, facial recognition
- Medical image analysis, autonomous vehicles
- Any task involving spatial/grid data

Strengths vs Weaknesses

✅ **Strengths:** Translation invariant, hierarchical feature learning, parameter sharing reduces overfitting, excellent for image data ❌ **Weaknesses:** Requires large datasets, computationally expensive, not rotation invariant, many hyperparameters

CNN vs Traditional Neural Networks

- **CNN:** Preserves spatial structure, local connectivity, weight sharing
- **Traditional NN:** Treats pixels independently, loses spatial information, more parameters

5. K-Means Clustering

What It Is

An **unsupervised** learning algorithm that partitions data into k clusters by minimizing within-cluster sum of squares.

How It Works: The K-Means Dance

Step 1: Initialize

- Choose number of clusters (k)
- Randomly place k centroids in feature space

Step 2: Assignment

- Assign each point to nearest centroid (using distance metric)
- Forms k clusters

Step 3: Update

- Move each centroid to center (mean) of its assigned points
- Recalculate cluster centers

Step 4: Repeat

- Repeat steps 2-3 until convergence (centroids stop moving)

Mathematical Foundation

Objective Function (minimize):

$$J = \sum_{i=1}^k \sum_{x \in C_i} ||x - \mu_i||^2$$

Where μ_i is centroid of cluster i

Centroid Update:

$$\mu_i = (1/|C_i|) \times \sum_{x \in C_i} x$$

Choosing Optimal k

Elbow Method:

1. Run k-means for different k values
2. Plot within-cluster sum of squares vs k
3. Look for "elbow" - point where improvement slows
4. Choose k at the elbow point

Silhouette Analysis:

- Measures how well points fit in their clusters
- Score between -1 and 1
- Higher = better clustering

Distance Metrics

- **Euclidean:** $\sqrt{\sum (x_i - y_i)^2}$ (most common)
- **Manhattan:** $\sum |x_i - y_i|$
- **Cosine:** Based on angle between vectors

Memory Aid: "DANCE"

- **D**ecide on k (number of clusters)
- **A**ssign points to nearest centroid
- **N**ew centroid = mean of assigned points
- **C**heck convergence, repeat if needed
- **E**lbow method to choose optimal k

Initialization Methods

- **Random:** Random placement of centroids
- **K-means++:** Smart initialization, reduces iterations
- **Multiple runs:** Run several times, pick best result

Use Cases

- Customer segmentation, market research, image segmentation
- Data compression, feature learning
- Organizing large datasets into meaningful groups

Strengths vs Weaknesses

✅ **Strengths:** Simple, fast, works well with spherical clusters, guaranteed convergence ❌ **Weaknesses:** Need to choose k, sensitive to initialization, assumes spherical clusters, sensitive to outliers

When K-Means Works Best

- Spherical/circular clusters
- Similar cluster sizes
- Well-separated clusters
- Continuous features

K-Means vs Other Clustering

- **vs Hierarchical:** Faster, but need to specify k
 - **vs DBSCAN:** Assumes spherical clusters, not density-based
-

6. K-Nearest Neighbors (KNN) 👤

What It Is

A **supervised**, lazy learning algorithm that classifies data points based on the majority class among k nearest neighbors in feature space.

Core Concept: "You Are Your Neighbors"

- Classification: Majority vote of k nearest neighbors
- Regression: Average of k nearest neighbor values
- No explicit training phase (lazy learning)

How It Works

1. **Store** all training data (no model building)
2. **For new point:** Calculate distance to all training points
3. **Find** k nearest neighbors
4. **Vote:** Classification = majority class; Regression = average value
5. **Predict** based on vote/average

Key Parameters

k (number of neighbors):

- **k = 1:** Very flexible, high variance, sensitive to noise
- **k = large:** Smoother decision boundary, may miss local patterns
- **k = odd:** Avoids ties in binary classification

Distance Metrics:

- **Euclidean:** $\sqrt{\sum (x_i - y_i)^2}$ (most common)
- **Manhattan:** $\sum |x_i - y_i|$ (good for high dimensions)
- **Minkowski:** Generalization of above
- **Hamming:** For categorical data

Choosing Optimal k

Cross-Validation Approach:

1. Try different k values (usually odd numbers)
2. Use cross-validation to test performance
3. Plot error rate vs k
4. Choose k with lowest error rate

Rule of Thumb: $k = \sqrt{n}$ (where n = number of training samples)

Memory Aid: "NEIGHBOR"

- **N**o explicit training (lazy learning)
- **E**uclidean distance commonly used
- **I**dentify k closest points
- **G**et majority vote for classification
- **H**igher k = smoother boundaries
- **B**ias-variance tradeoff with k
- **O**dd k prevents ties
- **R**equires feature scaling

Decision Boundaries

- **$k = 1$:** Very jagged, complex boundaries
- **$k = 3, 5$:** Moderately smooth
- **$k = \text{large}$:** Very smooth, may underfit

Lazy vs Eager Learning

- **Lazy (KNN):** No training phase, all work during prediction
- **Eager (most others):** Build model during training, fast prediction

Curse of Dimensionality

In high dimensions:

- All points become equidistant
- Nearest neighbors become less meaningful
- Performance degrades

Feature Scaling Critical!

- Different scales → distance dominated by large-scale features
- Always normalize/standardize features before using KNN

Use Cases

- Recommender systems, pattern recognition, outlier detection
- Simple baseline classifier
- Non-linear classification problems

Strengths vs Weaknesses

✔ **Strengths:** Simple, no assumptions about data, works with non-linear data, naturally handles multi-class ✖ **Weaknesses:** Computationally expensive, sensitive to irrelevant features, needs feature scaling, suffers in high dimensions

KNN vs Other Algorithms

- **vs Naïve Bayes:** No independence assumption, but slower
- **vs SVM:** Non-parametric, simpler, but less efficient
- **vs Decision Trees:** Smoother boundaries, less interpretable

Optimization Techniques

- **KD-Trees:** Fast neighbor search in low dimensions
- **LSH:** Locality-sensitive hashing for high dimensions
- **Approximate methods:** Trade accuracy for speed

🔄 Quick Comparison Table

Algorithm	Type	Best For	Key Strength	Main Weakness
Decision Trees	Supervised	Interpretability needed	Easy to understand	Prone to overfitting
Naïve Bayes	Supervised	Text classification	Fast, works with small data	Strong independence assumption
SVM	Supervised	High-dimensional data	Robust, versatile	Slow on large datasets
CNN	Supervised	Image processing	Spatial feature learning	Needs lots of data
K-Means	Unsupervised	Customer segmentation	Simple, fast	Need to choose k
KNN	Supervised	Simple baseline	No assumptions	Computationally expensive

🎯 Last-Minute Exam Tips

Formula Quick Reference

- **Entropy:** $E(S) = -\sum(p_i \times \log_2(p_i))$
- **Information Gain:** $IG = E(S) - \sum((|S_v|/|S|) \times E(S_v))$
- **Bayes:** $P(\text{class}|\text{features}) = P(\text{features}|\text{class}) \times P(\text{class}) / P(\text{features})$
- **SVM Margin:** $2/||w||$
- **CNN Output Size:** $(\text{Input} - \text{Filter} + 2 \times \text{Padding}) / \text{Stride} + 1$

Memory Tricks

- **Decision Trees:** "SPLIT" process
- **Naïve Bayes:** "BIAS" components
- **SVM:** "MARGIN" maximization
- **CNN:** "FILTER" for feature detection
- **K-Means:** "DANCE" algorithm steps
- **KNN:** "NEIGHBOR" voting system

Common Exam Question Types

1. **Algorithm steps:** Know the step-by-step process
2. **When to use:** Match algorithm to problem type
3. **Strengths/weaknesses:** Compare algorithms
4. **Parameter tuning:** k in KNN/K-means, C in SVM
5. **Mathematical foundations:** Key formulas and concepts

Good luck on your exam! 🍀